

실시간 렌더링 환경에서 Vulkan을 활용한 배치 렌더링 기반 드로우 콜 최소화

오정식⁰, 박정용, 이현규[†]
인천대학교 컴퓨터공학부
{ojs0331⁰, jylove0515, hyeonkyulee[†]}@inu.ac.kr

Draw Call Minimization via Batch Rendering In Real-time Rendering Using Vulkan

Jungsik Oh⁰, Jeongyong Park, Hyeonkyu Lee[†]
Department of Computer Science and Engineering, Incheon National University

요약

본 논문은 실시간 그래픽스 파이프라인에서 빈번하게 발생하는 드로우 콜(Draw Call)로 인해 발생하는 CPU-GPU 통신 병목을 최소화하기 위한 효율적인 배치 렌더링(Batch Rendering) 기법을 제안한다. 제안된 기법은 Vulkan의 Indirect 드로우 콜 기능을 활용하여 다수의 객체를 하나의 배치 단위로 결합하여 단 한 번의 드로우 콜 호출만을 수행함으로써 반복적인 드로우 콜 호출에 따른 명령어 송신 횟수를 획기적으로 절감하였다. 또한, FPS는 평균적으로 약 22.7% 향상되었다.

1. 서론

실시간 3D 그래픽스 엔진에서는 매 프레임마다 수천에서 수십만 건에 이르는 드로우 콜이 실행된다. 각 드로우 콜은 CPU가 GPU에게 렌더링과 관련된 명령 정보를 전달하는 단위로서, 드로우 콜이 많아질수록 CPU-GPU 간 통신 오버헤드가 기하급수적으로 증가한다[1]. 이러한 병목 현상은 고해상도 및 고주사율을 지향하는 현대 게임 및 시각화에서 전체적인 렌더링 성능을 결정짓는 주요 요인으로 작용한다.

본 연구에서는 Vulkan[2]의 Indirect 드로우 콜[3]을 활용하여 사전에 설정된 하이퍼 파라미터를 기준으로 다수의 객체들을 결합한 배치 단위로 만들고 Indirect, 정점(Vertex), 인덱스(Index) 버퍼들을 생성한다. 이를 기반으로 N 개의 드로우 콜을 단 하나의 Indirect 드로우 콜 호출로 통합 실행함으로써, 반복적인 명령 전달 횟수를 효과적으로 최소화한다. 이와 같은 최적화는 다수의 객체가 존재하는 복잡한 장면에서도 CPU-GPU 간 병목을 완화하고, 실시간 렌더링의 안정성과 성능을 확보하는 데 유효하다. 실제 실험에서는 제안 기법이 기존 렌더링 파이프라인 대비 드로우 콜 호출 횟수를 최대 97% 이상 절감하였으며, 평균 약 22.7%의 FPS(Frame Per Second) 향상을 달성하였으며, 이는 본

기법이 실효성이 있음을 보여준다.

2. 방법

2.1. 렌더링 파이프라인



그림 1: 배치 렌더링 파이프라인

본 연구에서 구성한 렌더링 파이프라인의 전체 절차를 그림 1에 제시하였다. 초기 배치 구축 단계는 오프라인에서 CPU에 의해 수행된다. 이 과정에서는 렌더링 대상 객체들을 사전 정의된 기준에 따라 배치 단위로 그룹화하고, Indirect 드로우 콜 호출을 위한 버퍼들을 생성한다.

2.2. 배치 및 Indirect 커맨드 버퍼 생성

배치 데이터 갱신 단계에서 CPU가 Vulkan의 Indirect 드로우 콜 호출에 필요한 다섯 개의 32비트 정수를 하나의 구조체 형태를 통해 작성한 후, 이를 임시 버퍼에 누적한다. 임시 버퍼의 크기가 사전에 설정된 하이퍼 파라미터 용량에 도달하면, 누적된 버퍼를 GPU에 전달하여 하나의 Indirect 커맨드 버퍼를 확정하고, 대응되는 정점 및 인덱스 데이터를 GPU 메모리에 할당한다.

기존 파이프라인에서는 메시(Mesh) M 개마다 상응하는 정점 및 인덱스 버퍼를 개별 할당되었으며, 반면 제안하는 배치 기반 접근은 이 과정을 배치 수 $N(N \ll M)$ 만큼으로 집약하여, 메모리 할당 호출 횟수를 $O(M)$ 에서 $O(N)$ 으로 축소시킨다. 이를 통해 GPU 드라이버 레벨에서의 메모리 관리 오버헤드가 효과적으로 감소되며, 전

* 포스터 발표논문
* 학부생 주저자 논문
† 교신저자

체 렌더링 효율성이 향상된다.

알고리즘 1: 배치 및 버퍼 생성 과정

```

1. procedure BUILD_BATCH(meshList, Capacity)
2.   batch ← new MiniBatch()
3.   for each mesh in meshList do
4.     vBytes ← mesh.vertexCount × sizeof(Vertex)
5.     iBytes ← mesh.indexCount × sizeof(uint32)
6.     if batch.size + vBytes + iBytes > Capacity then
7.       # 3개의 버퍼 생성 및 새로운 배치 생성
8.       createBuffers(batch)
9.       batch ← new MiniBatch()
10.    end if
11.    # Draw Commands 기록
12.    appendDrawCmd(batch, mesh)
13.    # 정점·인덱스 정보 누적
14.    appendData(batch, mesh.vertices, mesh.indices)
15.    batch.size ← batch.size + vBytes + iBytes
16.  end for
17.  # 배치의 잔여 데이터에 대한 버퍼 생성
18.  if batch.size > 0 then
19.    createBuffers(batch)
20.  end if
21. end procedure

```

배치에 포함된 모든 메시의 정점 및 인덱스 데이터는 단일 연속 메모리 블록으로 결합되어 VRAM에 저장된다. Indirect 커맨드 버퍼에 기록된 각 요소 i 는 동일 배치 내 정점 버퍼 및 인덱스 버퍼에서 특정한 오프셋 주소를 참조함으로써, 개별 메시의 렌더링에 필요한 메모리 영역을 정확히 지정할 수 있다.

2.3. Indirect 드로우 콜 호출

배치 기반 Indirect 렌더링의 핵심은 드로우 콜의 총 횟수를 최소화하는 데 있다. 제안한 알고리즘 1에 기술된 절차를 통해 생성된 N 개의 배치로 구성된다. 배치마다 한 번의 Indirect 드로우 콜을 호출만을 수행한다. 기존 방식이 메시 수(M)만큼 발생시키던 드로우 콜을 제안 기법은 배치 수($N \ll M$)로 압축한다. 드로우 콜 호출량은 $O(M)$ 에서 $O(N)$ 으로 감소된다.

3. 실험 및 결과

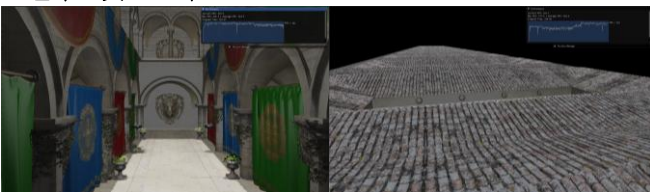


그림2: (좌)시점 변경 전 화면, (우)시점 변경 후 화면

삼각형 수($\times 10^3$)	기본(Basic) 렌더링 파이프라인	Ours
262	103	3
786	309	6
1,835	721	21
3,934	1,545	45

표 1: 드로우 콜 호출 횟수 비교 결과(배치 용량 설정=3MB)

성능 평가는 NVIDIA RTX 4060 Laptop GPU(8GB

VRAM)를 탑재한 환경에서 수행되었으며, 테스트 장면은 Sponza를 사용하였다. 평가 지표로는 (1) 측정된 FPS(Frame-Per-Second)[1], (2) 한 프레임에 호출된 드로우 콜 호출 횟수[3]를 이용하였다. 드로우 콜 횟수는 NVIDIA Nsight Graphics의 Frame Debugger로 캡처한 호출 횟수를 집계하였다. 제안하는 배치 기반 Indirect 파이프라인은 동일 장면을 단 3회, 6회, 21회, 45회의 Indirect 드로우 콜만으로 렌더링해 드로우 콜을 최대 97% 이상 절감하였음을 표 1에서 입증하였다. 단일 메모리 주소를 참고해 렌더링하므로, 기본과 제안한 방식에 있어서 렌더링 퀄리티 차이가 없다. 그림 3의 위 그래프는 그림 2의 시점 변경 전 화면에서, 삼각형 수가 증가함에 따라 FPS 변화를 나타낸 것이다. 삼각형 수가 약 15배 증가함에 따라, 기본 방식은 FPS가 급격히 하락하여 133에서 32로 크게 감소한 반면, 제안한 배치 방식은 동일한 조건에서 FPS가 163에서 119로 비교적 완만하게 감소하여 성능 감소폭을 현저히 억제하였다. 아래 그래프에서는 실시간 카메라 시점 변경에 따른 FPS 비교에서 제안한 방식은 이전보다 높은 FPS를 유지하는 반면, 기본 방식은 시점 변경 후에도 FPS가 동일한 상태를 유지하고 있다. 그림 3을 통해, 제안하는 배치 방식은 FPS가 더 높고, 변화에 대해 더 안정적인 성능을 유지하므로, 성능 향상과 성능 안정성이 동시에 개선되었음을 알 수 있습니다.

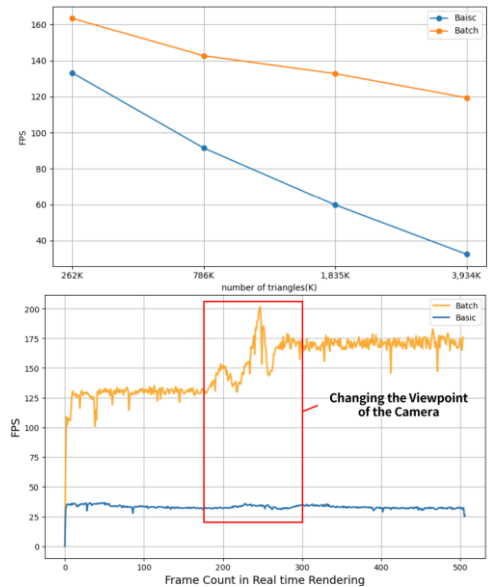


그림 3: (상)기본 및 배치 파이프라인 평균 FPS 실험 결과, (하)삼각형 3,934K 환경에서 기본 및 배치 파이프라인 실시간 FPS 실험 결과

참고문헌

- [1] Matthias Wloka, "Batch, Batch, Batch:" What Does It Really Mean?, Game Developers Conference, San Jose, CA, USA, 2003. <https://www.nvidia.com/docs/IO/8230/BatchBatchBatch.pdf>
- [2] Tristan Lorach, Vulkan: the essentials, Game Developers Conference, San Francisco, CA, USA, 2016. https://developer.download.nvidia.com/gameworks/events/GDC2016/Vulkan_Essentials_GDC16_tlorach.pdf
- [3] Christoph Kubisch and Matthias Niessner, GPU Driven Rendering Pipelines, SIGGRAPH, Los Angeles, CA, USA, 2015.